Decentralized Application for Shipment Services

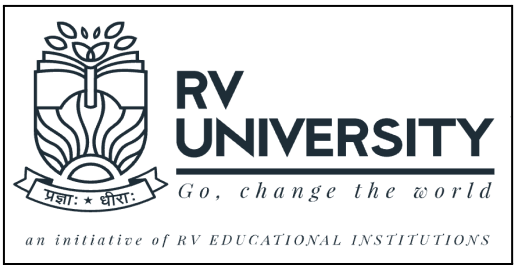
A PROJECT REPORT
Submitted by

Aadya Arun Talreja [1RVU23CSE003]
Amogha V Prasad [1RVU23CSE046]
Freya D Raja [1RVU23CSE158]

as part of the ***Continuous Internal Evaluation***
for the course

Advanced Blockchain Technologies

Offered in
Semester IV
of
B.Tech (Hons)



School of Computer Science and Engineering
RV University
RV Vidyaniketan, 8th Mile, Mysuru Road, Bengaluru, Karnataka, India - 562112

APRIL 2025

DECLARATION

I, **Aadya Arun Talreja (1RVU23CSE003), Amogha V Prasad (1RVU23CSE046) and Freya D Raja (1RVU23CSE158)** students of **fourth** semester B.Tech in **Computer Science & Engineering,** at School of Computer Science and Engineering, **RV University,** hereby declare that the project work titled "**Indian Personal Finance and Spending Habits**" has been carried out by us and submitted in partial fulfilment for the award of degree in **Bachelor of Technology in Computer Science & Engineering** during the academic year **2024-2025**. Further, the matter presented in the project has not been submitted previously by anybody for the award of any degree or any diploma to any other University, to the best of our knowledge and faith.

Name: Aadya Arun Talreja Signature
USN: 1RVU23CSE003

Name: Amogha V Prasad Signature
USN: 1RVU23CSE046

Name: Freya D Raja Signature
USN: 1RVU23CSE158

Place: RV University

Date: 17/04/2025

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

School of Computer Science and Engineering

RV University
RV Vidyaniketan, 8th Mile, Mysuru Road, Bengaluru, Karnataka, India – 562112

CERTIFICATE

This is to certify that the project work title **"Decentralized Application for Shipment Services"** is performed by **Aadya Arun Talreja (1RVU23CSE003), Amogha V Prasad (1RVU23CSE046) and Freya D Raja (1RVU23CSE158)** , a bonafide students of Bachelor of Technology at the School of Computer Science and Engineering, RV university, Bangalore as part of the **CIE-3 component** for the course **AI-Powered Business Intelligence** offered in Semester IV of Bachelor of Technology in Computer Science & Engineering , during the Academic year **2024-2025**.

Name of Course Faculty: Professor Sathya D

Signature with date of Faculty

TABLE OF CONTENTS

1. Problem Statement

In today's fast-paced digital commerce environment, traditional shipment systems face several pressing issues:

- ➔ Lack of transparency in the order fulfillment and shipping lifecycle
- ➔ Centralized control, which makes systems vulnerable to tampering and data breaches
- ➔ Complex dispute resolution due to lack of verifiable records
- ➔ Manual and error-prone order tracking, especially in cross-border and multi-vendor scenarios
- ➔ Customer dissatisfaction due to poor visibility into the status of their orders

These inefficiencies not only impact operational costs but also damage the trust relationship between vendors and customers.

To resolve these, we need a decentralized, secure, and tamper-proof system that:

- ➔ Enables transparent tracking of shipment lifecycle
- ➔ Automates business logic and customer interactions via smart contracts
- ➔ Eliminates the need for intermediaries
- ➔ Ensures trustless confirmation through cryptographic techniques like OTP verification

2. Introduction

The Shipment Service project is a blockchain-powered decentralized application (DApp) that enables users to manage and track orders through a smart contract deployed on the Ethereum blockchain. The DApp combines:

- Smart contract logic in Solidity for order lifecycle automation
- A React-based web interface for interaction
- MetaMask and Web3.js/Ethers.js for secure blockchain connectivity

This system empowers customers to:

- Create orders
- Track their order status across predefined stages
- Securely confirm delivery using a One-Time Password (OTP)
- Cancel orders in a self-service manner if still unfulfilled

The decentralized nature of this application makes it highly resistant to manipulation or data loss. It creates a verifiable, trustless ecosystem where every state change is immutably recorded on the blockchain.

3. Tools and Technologies Used

3.1 Development Stack

Layer	Tool/Technology	Purpose
Frontend	React.js	Build UI and handle user interactions
	HTML/CSS/JS	Presentation layer and dynamic rendering
Smart Contracts	Solidity	Define business logic on Ethereum
Blockchain Integration	Web3.js or Ethers.js	Connect frontend to Ethereum
Wallet	MetaMask	User wallet to sign transactions
Development Environment	Hardhat	Compile, test, and deploy smart contracts
Testing Blockchain	Chai	Local Ethereum simulation
Deployment	Ethereum Testnet / Hardhat Testnet	Hosting the smart contract

3.2 Additional Tools (Optional/Future Use):

- IPFS for decentralized storage of invoices or shipping documents
- The Graph for indexing and querying contract data
- Chainlink Oracles to integrate with off-chain logistics providers

4. Implementation Details

4.1 Smart Contract Architecture

The Shipment Service project is built using the Solidity programming language, running on the Ethereum Virtual Machine (EVM). The smart contract encapsulates the logic for an end-to-end shipment and order management system on the blockchain. It offers a decentralized, transparent, and tamper-proof mechanism for handling customer orders, payments, order tracking, and refunds.

4.1 Contract Initialization and Ownership

At deployment, the contract initializes the `owner` state variable to the address of the deploying entity (`msg.sender`). This address is granted exclusive privileges to perform specific administrative tasks like marking orders as shipped or delivered, and generating OTPs for secure delivery confirmation. This access is controlled using the `onlyOwner` modifier.

```
address public owner;
constructor() {
    owner = msg.sender;
}
```

The `onlyOwner` modifier is defined to restrict certain functions:

```
modifier onlyOwner() {
    require(msg.sender == owner, "Only the Owner can perform this
action");
    _;
}
```

4.2 Order Lifecycle Management

The contract defines a well-structured lifecycle for every order using the `OrderStatuses` enum:

```
enum OrderStatuses { Invalid, Processing, Shipped, Delivered,
Canceled }
```

Each order moves through various states—starting from `Processing`, then optionally transitioning to `Shipped`, `Delivered`, or `Canceled`.

4.3 Creating an Order

Customers interact with the system primarily through the createOrder() function. This function:

- Accepts item name and quantity as input.
- Calculates the total cost using the calculateOrderCost() function.
- Requires payment via msg.value that must cover or exceed the order cost.
- Transfers payment to the owner.
- Stores relevant order data using multiple mappings for tracking.

```
function createOrder(string memory _item, uint _quantity) public
payable onlyCustomer returns(uint256)
```

Each order is assigned a unique orderNumber and stored in several mappings for future reference. It also emits the OrderCreated event for off-chain tracking.

The calculateOrderCost() function assigns a price based on the item type using keccak256 hashing for comparison, which is essential since string comparison in Solidity is expensive and limited:

```
function calculateOrderCost(string memory _item, uint _quantity)
internal pure returns (uint)
```

Item prices:

- Fruit: 1 wei per unit
- Furniture: 3 wei per unit
- Computer: 4 wei per unit
- Others: 2 wei per unit

4.4 Shipping and OTP Generation

To mark an order as shipped, the contract owner uses ShipAndCreateOtp(). This function:

- Checks that the order exists and is not already delivered or shipped.
- Verifies that a valid OTP (between 999 and 9999) is provided.
- Updates the order status to Shipped.
- Stores the OTP associated with the order.

```
function ShipAndCreateOtp(uint256 _orderNumber, uint otp) public
onlyOwner
```

This OTP is crucial for confirming delivery securely and is only visible to the customer via getOTP():

```
function getOTP(uint256 _orderNumber) public view onlyCustomer
returns (uint)
```

4.5 Order Acceptance (Delivery Confirmation)

Once the customer receives the order, they must confirm delivery using `acceptOrder()`. They need to provide the correct OTP associated with the order. If the OTP matches, the order is marked as `Delivered`.

```
function acceptOrder(uint256 _orderNumber, uint otp) public
onlyCustomer
```

This step ensures that delivery acknowledgment is cryptographically validated and prevents unauthorized claims of delivery.

4.6 Order Cancellation and Refunds

Customers have the ability to cancel their orders if the order status is still `Processing`. The `cancelOrder()` function:

- Validates ownership of the order.
- Ensures the order is not already shipped or delivered.
- Calculates a refund amount (90% of the original order cost).
- Transfers the refund to the customer's wallet.
- Updates the order status to `Canceled`.

```
function cancelOrder(uint256 _orderNumber) public payable
onlyCustomer
```

The remaining 10% is retained as a cancellation fee, which may represent administrative or processing costs.

4.7 Data Structures and Mappings

To efficiently store and retrieve order data, the contract uses several Solidity mappings:

- `customerOrders`: Maps each order number to the customer's address.
- `itemsForOrder`: Maps order number to an array of item names.
- `quantitiesOfItemForOrder`: Nested mapping to get quantity of each item per order.
- `orderStatus`: Tracks the current status of each order.
- `orderOTP`: Stores the OTP for confirming deliveries.
- `orderCosts`: Keeps track of the total order value.
- `orderCancellationRefunds`: Records the refund amount (useful for audits).

These mappings enable efficient access to customer-specific and order-specific data.

4.8 Administrative Functions

To help the contract owner monitor logistics, several getter functions are included:

- getShippedOrders() – Returns all order numbers currently marked as shipped.
- getProcessingOrders() – Returns all orders under processing.
- getDeliveredOrders() – Returns all delivered orders.
- getOrdersForAddress(address) – Returns all orders placed by a specific customer.

These functions scan through all existing orders (1 to orderNumber) and extract those matching the desired status or address.

4.9 Customer-Facing Functions

Customers are given read-only access to retrieve data about their orders:

- getOrderStatus() – Get current status of any order.
- getItemsForOrder() – Retrieve item list for a given order number.
- getQuantitiesOfItemForOrder() – Get quantity of a specific item in an order.

These functions help customers stay informed about the status and contents of their orders in real time, with all data stored immutably on the blockchain.

4.10 Security Considerations

- Access Control: Use of modifiers (onlyOwner, onlyCustomer) ensures role-based access.
- Payment Verification: Orders cannot be created without adequate Ether.
- Delivery Confirmation via OTP: Prevents false delivery confirmations.
- Data Integrity: All records are permanently stored and tamper-proof.
- Refund Handling: Ensures a fair policy for customers canceling orders

4.2 Frontend Implementation (React)

The frontend in `frontend/src/` serves as the user gateway to interact with the Ethereum blockchain.

4.2.1 Features

1) Order form with item and quantity input
2) Order history panel with status labels
3) Accept Order panel with OTP input
4) Cancel Order functionality with real-time validation
5) MetaMask wallet connection logic
6) State updates upon transaction confirmation

4.2.2 Frontend Logic Highlights

- React Hooks for managing state and UI lifecycle
- Asynchronous JavaScript functions to handle transaction flow
- ABI (contract interface) and deployed address integration (expected in `constants.js` or `.env`)
- Alerts and notifications based on transaction events

4.2.3 User Journey

1. Connect Wallet → Create Order
2. Admin Marks Order as Shipped → OTP sent
3. Customer Accepts Order using OTP
4. Or Cancels it if within permissible window

4.3 Testing and Deployment

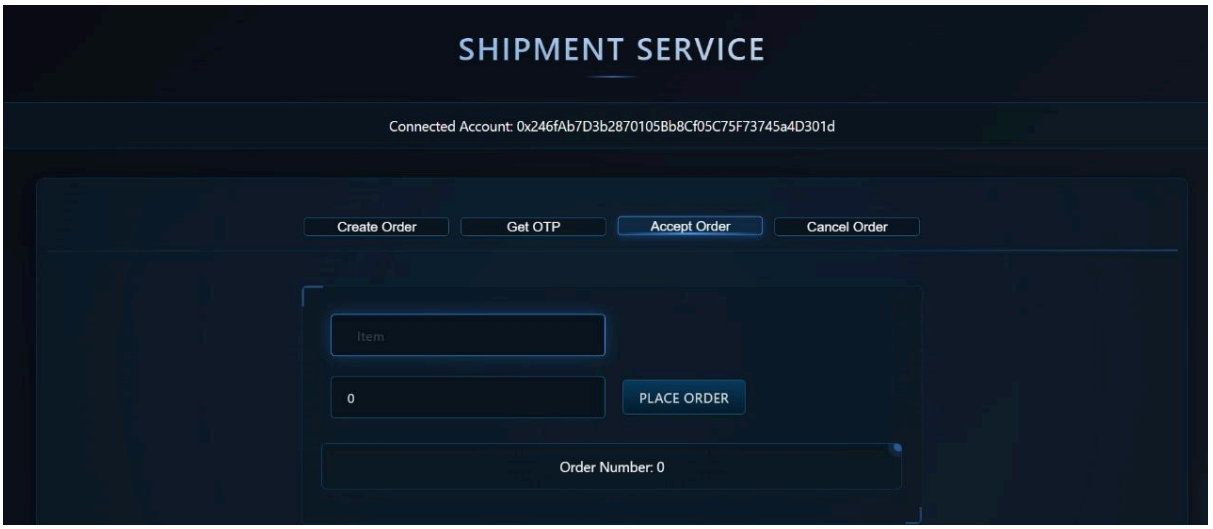
4.3.1 Testing Use Cases:

- Creating multiple orders and verifying state
- Handling invalid OTPs or unauthorized interactions
- Trying to cancel an already shipped order
- Verifying refund logic
- Simulating various network conditions.

4.3.2 Deployment (Expected Workflow):

- Compile contract: `npx hardhat compile`
- Deploy: `npx hardhat run scripts/deploy.js --network sepolia`
- Record contract address and ABI
- Plug into frontend for interaction

5. Results and Discussion



5.1 Achievements:

- The DApp delivers a self-sustained order tracking and management system
- All transactions are immutably recorded on-chain
- OTP ensures only the rightful recipient accepts the order
- Order state is instantly visible and traceable from creation to delivery

5.2 Technical Benefits:

- Eliminates central server vulnerabilities
- Reduces cost of intermediaries
- Provides cryptographic trust and transparency
- Seamlessly integrates financial logic (refunds) into workflow

5.3 Limitations and Challenges:

- Lack of zero-knowledge proofs for OTP handling — potential for brute-force if not rate-limited
- Gas costs may spike under high traffic or with large item sets
- No native mobile support (requires MetaMask)
- Contract file and complete frontend code need cleanup or restructuring for production

6. Conclusion

The Shipment Service DApp is a robust demonstration of how decentralized technologies can revolutionize supply chain and logistics. By leveraging Ethereum smart contracts, the system offers:

- Trustless Order Fulfillment: No need for intermediaries to confirm delivery.
- Transparent Tracking: Each step in the shipping process is recorded and verifiable.
- Secure Delivery: OTP-based system reduces fraud.
- Automated Refunds: Smart contract logic ensures fair handling of cancellations.

With enhancements such as secure random number generation, integration of oracles, and layer-2 scalability solutions, the platform can evolve into a powerful logistics tool fit for real-world deployments across industries such as e-commerce, warehousing, and international shipping.